

Technical Design for Deploying the Prediction Model through an Online Endpoint

The goal of this proposal is to deploy the trained XGBoost model as a prediction service accessible via an online API. The system will ingest shipment data, process it, and return predictions indicating the likelihood of delivery delays. This proposal primarily focuses on an AWS-based infrastructure, leveraging its cloud-native services for scalability, efficiency, and ease of deployment. However, equivalent solutions and services are available from other major cloud providers, such as Google Cloud and Microsoft Azure, and can be adapted to fit their respective environments.

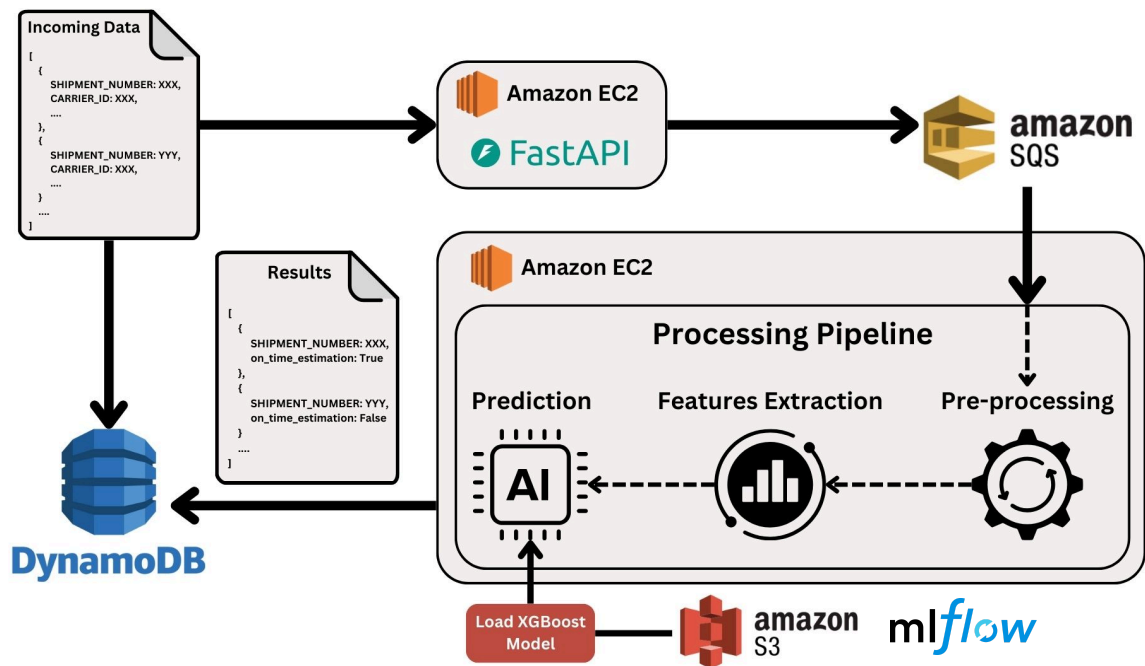
Components

1. Model Serving Infrastructure

- **API:** An AWS EC2 instance running FastAPI will serve as the entry point, accepting incoming shipment data and initiating predictions.
- **Model Persistence:** The trained model will be stored in cloud storage (e.g., AWS S3) and loaded into memory at startup to ensure it is ready for fast, real-time inference.
- **Queueing Jobs:** Shipment requests will be sent to an AWS SQS FIFO queue, enabling scalable, sequential processing.
- **Inference Engine:** Hosted on an EC2 instance, the inference engine loads the XGBoost model into memory and listens for incoming shipment data from the queue. It processes the data, extracts relevant features, and returns predictions.

Note 1: As XGBoost is computationally efficient, it does not require a GPU instance for inference. A well-configured CPU instance is sufficient, with the option to scale up if needed.

Note 2: If the data load is small, the API and the Inference Engine can be hosted on the same machine to simplify deployment and reduce operational overhead. The following diagram illustrates a high-level AWS-based infrastructure suggestion for this setup.



2. API

The API will accept shipment data in JSON format, containing key features such as carrier ID, vehicle size, postcodes, and collection/delivery times. This data can optionally be stored in a NoSQL database such as DynamoDB for record-keeping and tracking purposes.

Upon receiving the shipment data, the API will place the requests into a queue (e.g., AWS SQS). The inference engine will retrieve these queued requests, process them by extracting relevant features, and use the pre-loaded model for prediction.

Note: Preprocessing tasks, such as handling missing data, correcting GPS information, and extracting features, can potentially be performed by the API before submitting data to the queue. Whether this preprocessing happens at the API stage or the inference stage depends on available resources and the design of the system.

3. Feature Engineering Pipeline

The feature engineering pipeline can run either on the same machine as the API or on the machine handling predictions. This pipeline processes shipment data from the queue and applies the same preprocessing steps used during model training (e.g., filling missing data, correcting GPS errors, extracting key features). Ensuring that the same preprocessing is applied to the test data guarantees consistent predictions.

4. Prediction

For making predictions, the XGBoost model will be loaded from storage (e.g., AWS S3) into memory, allowing fast access for real-time predictions. Once the data has been preprocessed and features extracted, the features will be passed to the model for inference. The prediction results, formatted as a JSON object with two keys (Shipment ID and On-Time Delivery

probability), will then be stored in a NoSQL database such as DynamoDB for further use or reporting.

5. Model Tracking and Management - MLflow

Instead of simply storing the model on S3, model tracking software can enhance performance monitoring, version control, and deployment. Industry-standard tools like MLflow provide streamlined management, allowing the team to test new versions trained on additional data or roll back to previous versions if needed. MLflow has the option to store and load models from/to S3.

6. Scalability & Availability

- **Containerisation:** The API and processing pipeline can be packaged using Docker to ensure consistency across different environments.
- **Load Balancing:** We can use a load balancer (e.g., **AWS ELB**) to distribute requests across multiple instances.
- **Auto-scaling:** Enable auto-scaling (e.g., **AWS EC2 Auto-scaling**) to dynamically adjust to varying loads.

5. Monitoring & Logging

- **Logging:** All incoming requests and prediction outcomes will be logged for debugging and monitoring purposes.
- **Monitoring:** **Prometheus** or **Grafana** can be used to track API health, performance, and error rates.
- **Model Performance:** Predictions will be compared with actual shipment outcomes over time to assess model performance.

6. Security

- **Authentication:** Implement token-based authentication (e.g., OAuth2, JWT) to secure the API.
- **HTTPS:** Enforce secure communication using **SSL/TLS**.
- **Rate Limiting:** Apply rate limiting to manage API usage and prevent overload.